

SoK: Computer-Aided Cryptography

KeyWord: Computer-aided cryptography, Design-Level Security, Symbolic Tool, Computational Tool, Functional Correctness and Efficiency

Ji, Yonghyeon

hacker3740@kookmin.ac.kr

Department of Cyber Security
Kookmin University



CSE

**CRYPTO & SECURITY
ENGINEERING Lab**
암호 및 보안 공학 연구실

April 29, 2026

Contents





I. Introduction



2021 IEEE Symposium on Security and Privacy (SP)

SoK: Computer-Aided Cryptography

Manuel Barbosa^{*}, Gilles Barthe^{†‡}, Karthik Bhargavan[§], Bruno Blanchet[§], Cas Cremers[¶], Kevin Liao^{†||}, Bryan Parno^{**}

^{*}University of Porto (FCUP) and INESC TEC, [†]Max Planck Institute for Security & Privacy, [‡]IMDEA Software Institute,

[§]INRIA Paris, [¶]CISPA Helmholtz Center for Information Security, ^{||}MIT, ^{**}Carnegie Mellon University

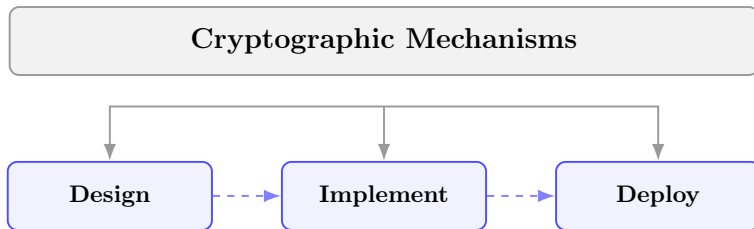
I. Introduction

Problem, Prior, and Approach/Direction



- **Problem:**

Cryptographic mechanisms are **hard to design, implement, and deploy correctly (securely)**, and often fail through flawed proofs, bugs, or side-channel leakage.



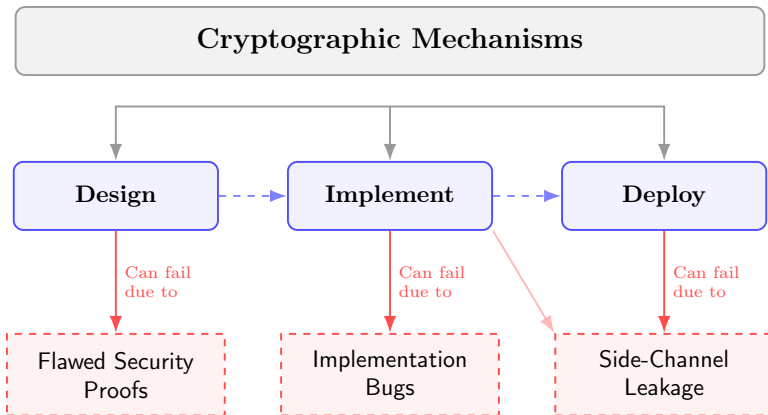
I. Introduction

Problem, Prior, and Approach/Direction



- **Problem:**

Cryptographic mechanisms are **hard to design, implement, and deploy correctly (securely)**, and often fail through flawed proofs, bugs, or side-channel leakage.



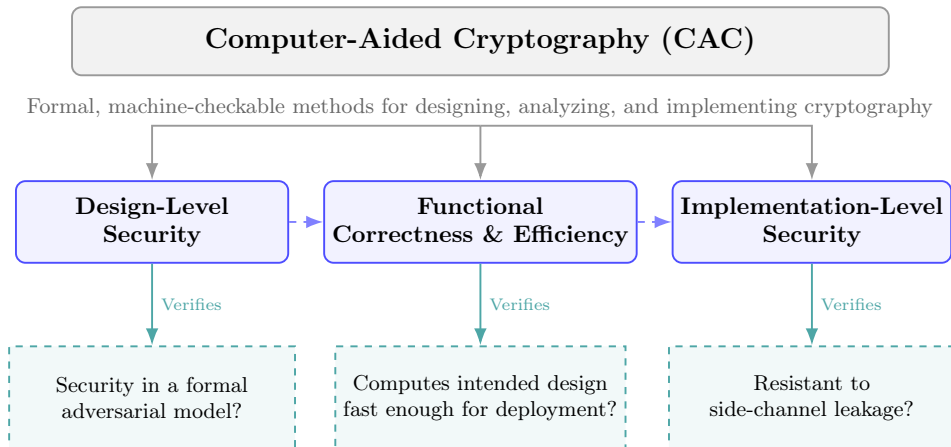
I. Introduction

Approach



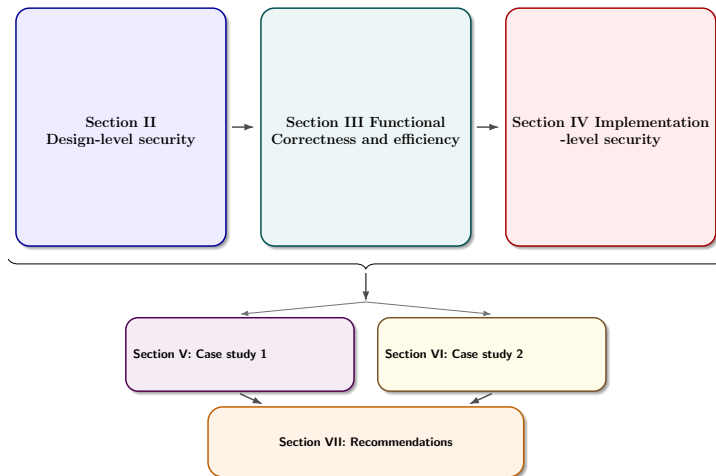
- **Approach:**

The paper systematizes **computer-aided cryptography (CAC)** across three layers:



I. Introduction

Structure of the Paper



I. Introduction

How to evaluate several tools?



Accuracy (A) How strong/accurate the guarantee is?

Scope (S) What the tool can express and analyze?

Trust (T) How much confidence we have in the result?

Usability (U) How practical the tool is for users ?



II. Design-Level Security

II. Design-Level Security

The Two Design-Level Paradigms



Design-level security have been developed in two largely separate communities:

- **Symbolic security** (in the formal methods community)
- **Computational security** (in the cryptography community).

II. Design-Level Security

The Two Design-Level Paradigms



Aspect	Symbolic Security	Computational Security
Main community	Formal methods	Cryptography
Main object	Protocol logic	Cryptographic security theorem
Messages	Abstract terms	Bitstrings
Cryptographic primitives	Idealized symbolic functions	Probabilistic algorithms
Adversary	Symbolic Dolev–Yao style adversary	Probabilistic polynomial-time adversary
Security style	Logical reachability or indistinguishability	Game-based or simulation-based proof
Main strength	Automation and attack discovery	Cryptographic soundness and quantitative guarantees
Typical tools	ProVerif, Tamarin	CryptoVerif, EasyCrypt



Basic model. In symbolic security, messages are formal terms. For example:

$$m, \quad k, \quad \text{Enc}(m, k), \quad \text{Dec}(c, k).$$

Cryptographic primitives are treated as **black-box functions**. For symmetric encryption, one may write:

$$\text{Dec}(\text{Enc}(m, k), k) = m.$$

Adversary model. A symbolic adversary is usually a Dolev–Yao style attacker. It can

- intercept messages;
- replay messages;



Security properties. Symbolic security commonly proves two kinds of properties.

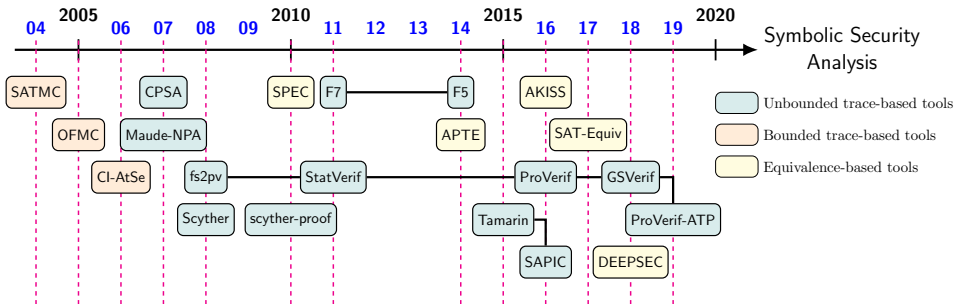
- (1) **Trace properties.** A trace property says:
“a bad event never occurs in any execution trace.”
→ These are natural for secrecy and authentication.
- (2) **Equivalence properties.** An equivalence property says:
“two protocol executions are indistinguishable to the adversary.”
→ This is used for privacy-style goals such as anonymity and vote privacy.

Strengths of symbolic security. Symbolic security is powerful when the question is:

“Is there a logical flaw in this protocol design?”

II. Design-Level Security - Symbolic Tools

Timeline of Tools for Symbolic Security Analysis



	Tool family	Explanation
One-world	Unbounded trace Bounded trace	Can we analyze an unbounded #(protocol sessions ¹)? Do not consider attacks beyond the cut-off.
Two-world	Equivalence	Does support verification of equivalence properties?

¹sessions = one run of the protocol

II. Design-Level Security - Symbolic Tools

A/S/T/U for Symbolic Tools in Design-Level Security



Accuracy (A) 1. Unbounded Number of Sessions

- A protocol may be secure for (N) sessions but insecure for $(N + 1)$ or more.
- An unbounded-session tool gives a stronger, more accurate security claim.

- ### **Scope (S)**
1. Trace Properties
 2. Equivalence Properties
 3. Equational Theories
 4. Global Mutable State

- ### **Trust (T)**
1. Link to Implementation
 2. Independent Verifiability

- ### **Usability (U)**
1. Abstractions
 2. Interactive Mode
 3. Specification Language

II. Design-Level Security - Symbolic Tools

A/S/T/U for Symbolic Tools in Design-Level Security



Scope (S) 1. Trace properties

- Can it prove that bad events never occur,
- e.g. “the adversary never learns the session key”?

2. Equivalence properties

We want to compare two protocols (or two variants of one protocol):

$$P_0 \quad \text{vs} \quad P_1.$$

Security means that

“Adversary $\mathcal{A}$ cannot distinguish P_0 from P_1 ”.

II. Design-Level Security - Symbolic Tools

A/S/T/U for Symbolic Tools in Design-Level Security



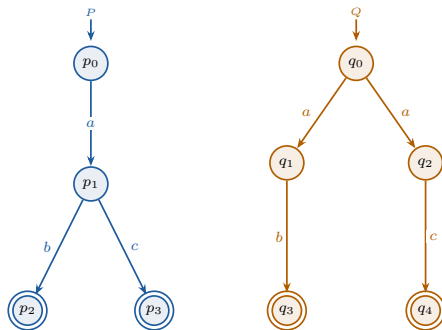
Scope (S) 1. Trace properties 2. Equivalence properties

- Trace equivalence (t)
- Open bisimilarity (o)
- Diff-equivalence (d)

- $\mathcal{T}(P) = \mathcal{T}(Q) = \{a, ab, ac, \varepsilon\}$
- $P \approx_t Q \iff$ same observable sequences
- But structure of branch differs

Trace equivalence

compares sets of traces



II. Design-Level Security - Symbolic Tools

A/S/T/U for Symbolic Tools in Design-Level Security



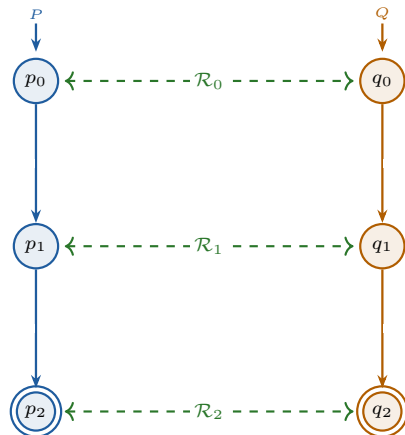
Scope (S) 1. Trace properties 2. Equivalence properties

- Trace equivalence (t)
- Open bisimilarity (o)
- Diff-equivalence (d)

- $P \approx_o Q \iff$ step-by-step ind.

Open bisimilarity

compares processes step by step



II. Design-Level Security - Symbolic Tools

A/S/T/U for Symbolic Tools in Design-Level Security



Scope (S) 1. Trace properties 2. Equivalence properties

- Trace equivalence (t)
- Open bisimilarity (o)
- Diff-equivalence (d)

Example (Real vs Ideal). Consider

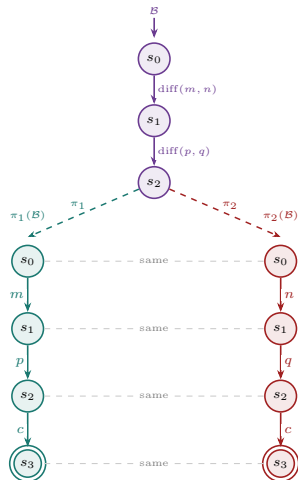
P_0 : send $\text{Enc}(s, k)$ P_1 : send $\text{Enc}(r, k)$

where s is the secret and r is a random value.
If the adversary cannot distinguish $\text{Enc}(s, k)$
and $\text{Enc}(r, k)$,

$$P_0 \approx_d P_1.$$

Diff-equivalence

two synchronized projections of one biprocess



II. Design-Level Security - Symbolic Tools

A/S/T/U for Symbolic Tools in Design-Level Security



- Scope (S)**
1. Trace properties
 2. Equivalence properties

Equivalence Properties	Strength	Expressiveness	Automation
Trace equivalence	weakest	highest	hardest
Open bisimilarity	medium	medium	moderate
Diff-equivalence	strongest	lowest	easiest

The hierarchy is

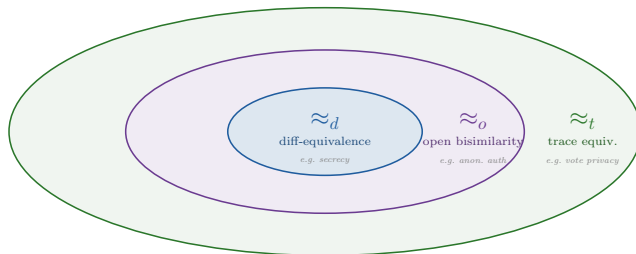
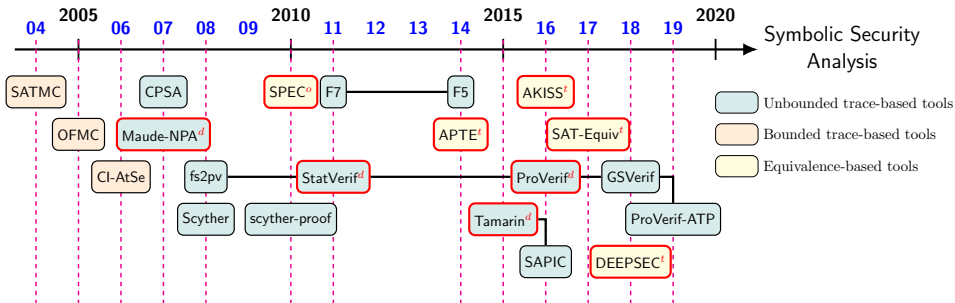
diff-equivalence $\implies$ open bisimilarity $\implies$ trace equivalence.

In other words,

$$\approx_d \subseteq \approx_o \subseteq \approx_t .$$

II. Design-Level Security - Symbolic Tools

A/S/T/U for Symbolic Tools in Design-Level Security



II. Design-Level Security - Symbolic Tools

A/S/T/U for Symbolic Tools in Design-Level Security



Scope (S) 3. Equational theories

- How much cryptographic algebra the tool can faithfully model?
- It is about the range of mathematical structures the tool can represent.

1. Fixed or no equational theories.

$$\text{dec}(\text{enc}(m, k), k) = m.$$

2. User-defined equations without associativity and commutativity.

$$\text{verify}(\text{sign}(m, sk), pk(sk)) = m.$$

3. User-defined equations with associativity and commutativity. XOR:

$$x \oplus y = y \oplus x, \quad (x \oplus y) \oplus z = x \oplus (y \oplus z).$$

For example, from m and $m \oplus k$, the adversary derives

$$m \oplus (m \oplus k) = (m \oplus m) \oplus k = k.$$

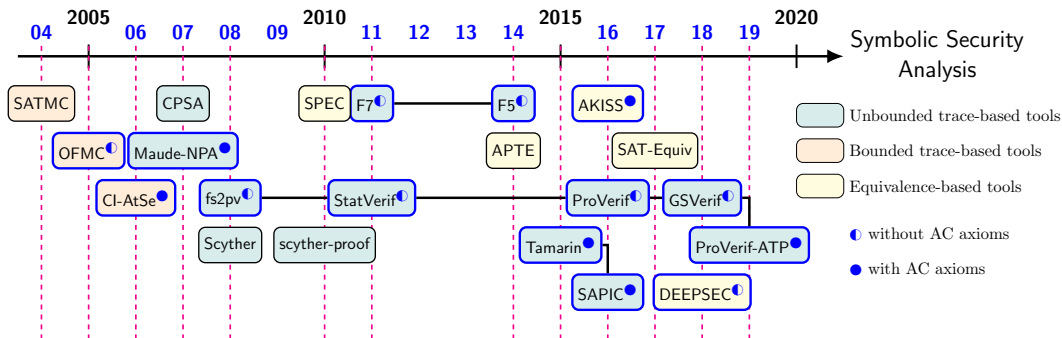
II. Design-Level Security - Symbolic Tools

A/S/T/U for Symbolic Tools in Design-Level Security



Scope (S) 3. Equational theories

→ AC: Associativity-Commutativity



4. Global mutable state

→ Whether the tool can model protocols with shared mutable state, such as databases, key servers, counters, or shared memory.

II. Design-Level Security - Symbolic Tools

A/S/T/U for Symbolic Tools in Design-Level Security



Trust (T) Link to implementation

- Can the tool extract/generate executable code from specifications in order to link symbolic security guarantees to implementations?
- F7, F5, fs2pv

Independent verifiability

- Symbolic tools implement complex algorithms that may contain bugs.
- Independent proof checking reduces the trusted computing base;
- scyther-proof², SPEC³

²S. Meier, C. J. F. Cremers, and D. A. Basin, “Strong invariants for the efficient construction of machine-checked protocol security proofs,” in IEEE Computer Security Foundations Symposium (CSF). IEEE Computer Society, 2010, pp. 231–245.

³A. Tiu and J. E. Dawson, “Automating open bisimulation checking for the spi calculus,” in IEEE Computer Security Foundations Symposium (CSF). IEEE Computer Society, 2010, pp. 307–321.

II. Design-Level Security - Symbolic Tools

A/S/T/U for Symbolic Tools in Design-Level Security



Usability (U) Abstractions

- Does the tool use abstraction?

Interactive mode

- Does the tool support an interactive analysis mode?

Specification language

- domain-specific security protocol languages
- process calculus
- multiset rewriting
- general programming language

II. Design-Level Security - Symbolic Tools

OVERVIEW OF TOOLS FOR SYMBOLIC SECURITY ANALYSIS



Tool		Unbound	Trace	Equiv	Eq-thy	State	Link
CPSA [▷]	[16]	●	●	○	○	●	○
F7 [◇]	[17]	●	●	○	●	●	●
ℒF5 [◇]	[18]	●	●	○	●	●	●
Maude-NPA [▷]	[19]	●	●	● ^d	●	○	○
ProVerif ^{*†}	[20]	●	●	● ^d	●	○	○
ℒfs2pv ^{◇†}	[21]	●	●	○	●	○	●
ℒGSVerif ^{*†}	[22]	●	●	○	●	●	○
ℒProVerif-ATP ^{*†}	[23]	●	●	○	●	○	○
ℒStatVerif ^{*†}	[24]	●	●	● ^d	●	●	○
Scyther [▷]	[25]	●	●	○	○	○	○
scyther-proof ^{▷‡§}	[26]	●	●	○	○	○	○
Tamarin ^{*‡}	[27]	●	●	● ^d	●	●	○
ℒSAPIC [*]	[28]	●	●	○	●	●	○
CI-AiSe [▷]	[29]	○	●	○	●	●	○
OFMC ^{▷†}	[30]	○	●	○	●	●	○
SATMC [▷]	[31]	○	●	○	○	●	○
AKISS ^{*†}	[32]	○	○	●	●	●	○
APTE [*]	[33]	○	○	● ^t	○	○	○
DEEPSEC [*]	[34]	○	○	● ^t	●	●	○
SAT-Equiv [*]	[35]	○	○	● ^t	○	○	○
SPEC ^{*.§}	[36]	○	○	● ^o	○	○	○

Specification language

- ▷ – security protocol notation
- * – process calculus
- * – multiset rewriting
- ◇ – general programming language

Miscellaneous symbols

- ℒ – previous tool extension
- † – abstractions
- ‡ – interactive mode
- § – independent verifiability

Equational theories (Eq-thy)

- – with AC axioms
- – without AC axioms
- – fixed

Equivalence properties (Equiv)

- t* – trace equivalence
- o* – open bisimilarity
- d* – diff equivalence

II. Design-Level Security - Symbolic Tools

Discussion: Achievements



Achievements Symbolic proofs for real-world case studies

- **ProVerif** and **Tamarin** stand out for scalability and expressivity.

Tool	Protocol	Time
ProVerif	TLS 1.0	seconds – hours
ProVerif	TLS 1.3	~1 hour
ProVerif	Signal	(minutes) – (>1 day)
Tamarin	5G authentication key exchange	~5 hours
Tamarin	TLS 1.3	~1 week, 100 GB RAM
Tamarin	DNP3 SAvS power grid	several minutes
Tamarin	Noise protocols	seconds – hours



Challenge Verifying Equivalence Properties

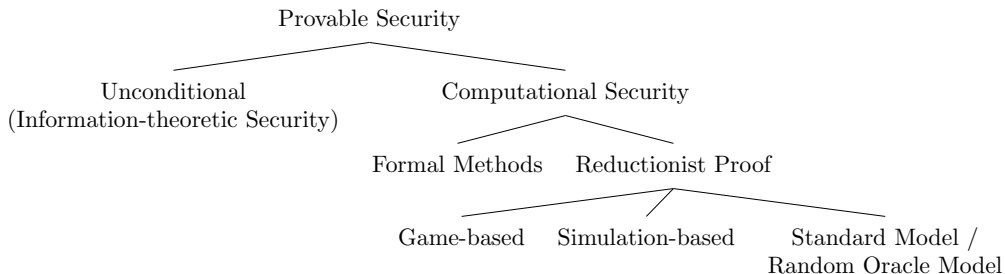
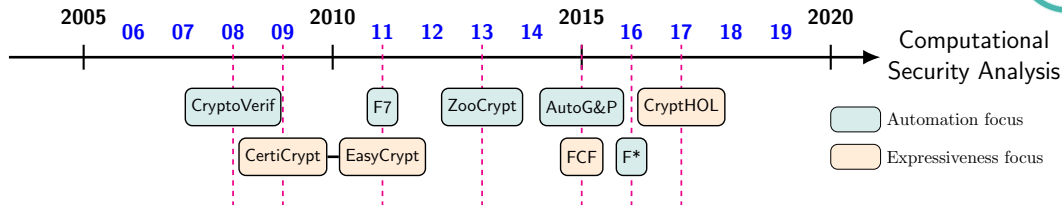
→ Equivalence properties are *inherently more difficult* to verify than trace properties — they involve **relations between traces**, not single traces.

For full automation, one must either:

- (1) **Bound the number of sessions** (loses the unbounded guarantee), or
 - (2) Use **diff-equivalence** — the strongest and most restrictive notion.
- Diff-equivalence **cannot** handle many desired properties.

II. Design-Level Security - Computational Tools

Timeline of Tools for Computational Security Analysis



II. Design-Level Security - Computational Tools

Design-level Computational Security



In **design-level computational security**,

- the tool is not just searching for symbolic attacks.
- It is helping prove a cryptographic theorem of the following form:

$$\Pr[A \text{ breaks scheme}] \leq \underbrace{\text{Adv}_R(\text{hard primitive})}_{\text{negligible by assumption}} + \underbrace{\varepsilon_{\text{stat}}}_{\text{statistical error}}$$

II. Design-Level Security - Computational Tools

A/S/T/U for Computational Tools in Design-Level Security



Accuracy (A) 1. Unbounded #(Sessions)

Scope (S) 1. Trace Properties
2. Equivalence Properties
3. Equational Theories
4. Global Mutable State

Trust (T) 1. Link to Implementation
2. Independent Verifiability

Usability (U) 1. Abstractions
2. Interactive Mode
3. Specification Language

Accuracy (A) 1. Concrete security

Scope (S) no separate criterion

Trust (T) 1. Link to Implementation
2. Trusted Computing Base

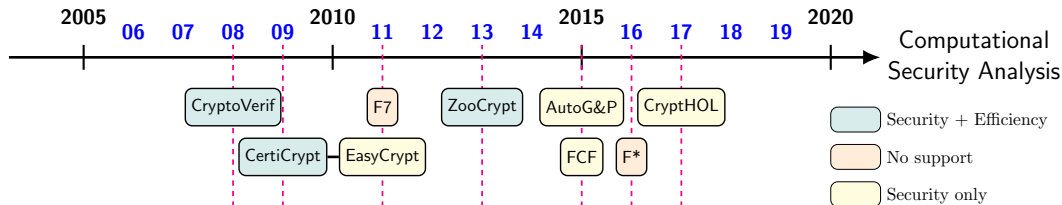
Usability (U) 1. Reasoning Focus
2. Automated Proof-finding
3. Composition
4. Specification Language

II. Design-Level Security - Computational Tools

A/S/T/U for Computational Tools in Design-Level Security



Accuracy (A) 1. Concrete security



Recall that

$$\Pr[A \text{ breaks scheme}] \leq \underbrace{\text{Adv}_R(\text{hard primitive})}_{\text{negligible by assumption}} + \underbrace{\varepsilon_{\text{stat}}}_{\text{statistical error}}$$

Example. For a MAC with tag length τ , one may want a theorem of the form

$$\text{Adv}_{\Sigma_{\text{PRF}}}^{\text{uf-cma}}(\mathcal{A}) \leq \text{Adv}_F^{\text{prf}}(\mathcal{B}) + \frac{q_{\text{ver}}}{2^\tau}$$

where q_{ver} is the number of forgery attempts.

II. Design-Level Security - Computational Tools

A/S/T/U for Computational Tools in Design-Level Security



Theorem (PRF-secure $\implies$ UF-CMA secure). Let $F : \mathcal{K} \times \mathcal{M} \rightarrow \{0, 1\}^\tau$ be a family of functions. For any UF-CMA adversary $\mathcal{A}$ against Σ_{PRF} asking q_{ver} queries (to $\mathcal{O}^{\text{MAC-Ver}}$), there is a PRF adversary against F such that

$$\text{Adv}_{\Sigma_{\text{PRF}}}^{\text{uf-cma}}(\mathcal{A}) \leq \text{Adv}_F^{\text{prf}}(\mathcal{B}) + \frac{q_{\text{ver}}}{2^\tau}.$$

Proof. Given $\mathcal{A}$ we build $\mathcal{B}$ as follows:

Adversary $\mathcal{B}$	Subroutine $\mathcal{O}^{\text{MAC-Gen}^*}(M)$
1. $S \leftarrow \emptyset, (M, T) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{MAC-Gen}^*}}, T' \leftarrow \text{Fn}(M)$ 2. if $T = T'$ and $M \notin S$ then $\text{res} \leftarrow 1$ else $\text{res} \leftarrow 0$ 3. return res	1. $T \leftarrow \text{Fn}(M)$ 2. $S \leftarrow S \cup \{M\}$ 3. return T

Here, $\text{Fn} \in \{F_K, \rho\}$ where $\rho \leftarrow \text{Func}(\mathcal{M}, \{0, 1\}^\tau)$. Since

$$\Pr[\text{Ideal}_F(\mathcal{B}) \Rightarrow 1] = \Pr[\rho(M_1) = T_1 \vee \dots \vee \rho(M_{q_{\text{ver}}}) = T_{q_{\text{ver}}}] \leq \sum_{i=1}^{q_{\text{ver}}} \Pr[\rho(M_i) = T_i] = q_{\text{ver}} \cdot 2^{-\tau},$$

we have

$$\text{Adv}_F^{\text{prf}}(\mathcal{B}) = \Pr[\text{Real}_F(\mathcal{B}) \Rightarrow 1] - \Pr[\text{Ideal}_F(\mathcal{B}) \Rightarrow 1] \geq \text{Adv}_{\Sigma}^{\text{uf-cma}}(\mathcal{A}) - \frac{q_{\text{ver}}}{2^\tau}. \quad \square$$

II. Design-Level Security - Computational Tools

A/S/T/U for Computational Tools in Design-Level Security



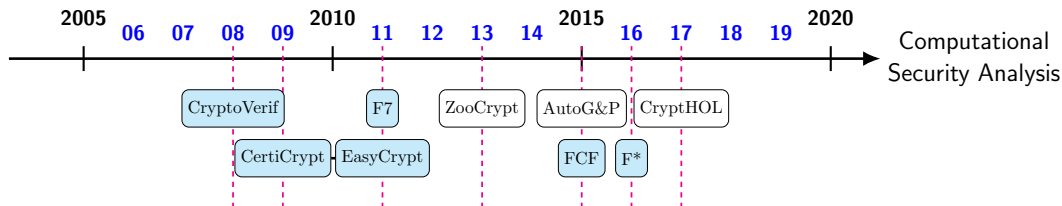
Accuracy (A) 1. Concrete security

Scope (S) **no separate criterion in the paper**

- **[CryptoVerif]** works ell for protocols such as key exchange,
- **[EasyCrypt]** closely follows pen-and-paper game-hopping arguments.

Trust (T) **1. Link to Implementation**

→ Whether the tool can extract, generate, or verify executable code from the specification



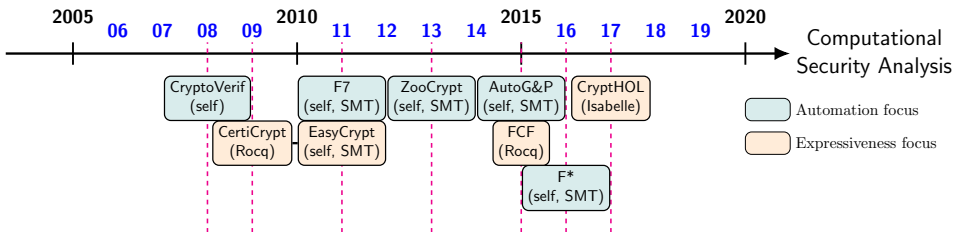
II. Design-Level Security - Computational Tools

A/S/T/U for Computational Tools in Design-Level Security



Trust (T) 1. Link to Implementation

2. Trusted Computing Base



	Isabelle	Rocq / Coq	SMT solvers
Type	Interactive theorem prover		Automated solver
Logic	Higher-order Logic (HOL)	CIC ⁴	First-order logic (FOL)

⁴Calculus of Inductive Constructions

II. Design-Level Security - Computational Tools

A/S/T/U for Computational Tools in Design-Level Security



Example 1 (Associative Law). Suppose we want to express that a binary operation is associative:

$$\forall x, y, z. (x \cdot y) \cdot z = x \cdot (y \cdot z).$$

FOL FOL handles this directly.

$$\forall x, \forall y, \forall z. (x \cdot y) \cdot z = x \cdot (y \cdot z).$$

HOL HOL also handles it directly.

$$\forall x, y, z : \alpha. \text{op}(\text{op}(x, y), z) = \text{op}(x, \text{op}(y, z)).$$

CIC CIC also handles it directly.

$$\begin{aligned} A &: \text{Type}, & \text{op} &: A \rightarrow A \rightarrow A, \\ \text{assoc} &: \forall x, y, z : A, \text{op}(\text{op}(x, y), z) = \text{op}(x, \text{op}(y, z)). \end{aligned}$$

II. Design-Level Security - Computational Tools

A/S/T/U for Computational Tools in Design-Level Security



Example 2 (Natural numbers and Induction). For any subset $A \subseteq \mathbb{N}$,

$$\begin{cases} 0 \in A \\ \forall n \in \mathbb{N}, n \in A \Rightarrow S(n) = n + 1 \in A \end{cases} \implies A = \mathbb{N}.$$

FOL For every first-order formula $\varphi(n)$,

$$[[\varphi(0)] \wedge [\forall n(\varphi(n) \Rightarrow \varphi(Sn))]] \Rightarrow [\forall n, \varphi(n)].$$

FOL cannot quantify over arbitrary predicates φ .

HOL $\forall P : \mathbb{N} \rightarrow \text{Prop.}, P(0) \Rightarrow (\forall n. P(n) \Rightarrow P(Sn)) \Rightarrow \forall n. P(n)$.

CIC $\forall P : \text{nat} \rightarrow \text{Type}, P(0) \rightarrow (\forall n, P(n) \rightarrow P(Sn)) \rightarrow \forall n, P(n)$.

II. Design-Level Security - Computational Tools

A/S/T/U for Computational Tools in Design-Level Security



Usability (U) 1. Reasoning Focus

- Automation-focused tools;
- Expressiveness-focused tools.

2. Automated Proof-finding

- Can the tool automatically find security proofs?
- *AutoG&P*: proofs of pairing-based schemes.
- *CryptoVerif*: proofs of key exchange protocols using a catalog of built-in game transformations.
- *ZooCrypt*: proofs of padding-based public key encryption schemes.

3. Composition

- Does the tool support compositional reasoning?
- *EasyCrypt*, *CryptHOL*, *F7*, *F**, *FCF*

II. Design-Level Security - Computational Tools

A/S/T/U for Computational Tools in Design-Level Security



- Usability (U)**
1. Reasoning Focus
 2. Automated Proof-finding
 3. Composition
 4. Specification Language

→ **Functional.** A functional language is natural for defining algorithms:

KeyGen, Enc, Dec or Hash, Sign, Verify.

It is good for primitives and functional correctness-style specifications.

→ **Imperative.** An imperative language is natural for game-based proofs like

$k \leftarrow \text{KeyGen}(); \quad b \xleftarrow{\$} \{0, 1\}; \quad c \leftarrow \text{Enc}_k(m_b); \quad b' \leftarrow \mathcal{A}(c); \quad \text{return } b = b'.$

II. Design-Level Security - Computational Tools

A/S/T/U for Computational Tools in Design-Level Security



Category	Criterion	Main question
Accuracy	Concrete security	Does the theorem give explicit probability and time bounds ?
Trust	Link to implementation	Does the proof apply to executable code , or only to an abstract model?
	Trusted Computing Base	What proof checker, solver, or tool implementation must be trusted ?
Usability	Reasoning focus	Is the tool focused for automation or expressiveness ?
	Automated proof-finding	Can the tool discover proof?
	Composition	Can component security theorems be combined modularly ?
	Specification language	Are protocols, games, and algorithms expressed in a natural language?

II. Design-Level Security - Computational Tools

OVERVIEW OF TOOLS FOR COMPUTATIONAL SECURITY ANALYSIS



Tool		RF	Auto	Comp	CS	Link	TCB
AutoG&P [◇]	[55]	●	●	○	●	○	self, SMT
CertiCrypt ^{▷◇}	[56]	●	○	○	●	●	Coq
CryptHOL [◇]	[57]	●	○	●	●	○	Isabelle
CryptoVerif ^{*◇}	[58]	●	●	○	●	●	self
EasyCrypt ^{▷◇}	[59]	●	○	●	●	●	self, SMT
F7 [◇]	[17]	●	○	●	○	●	self, SMT
F ^{*◇}	[60]	●	○	●	○	●	self, SMT
FCF [◇]	[61]	●	○	●	●	●	Coq
ZooCrypt [◇]	[62]	●	●	○	●	○	self, SMT

Reasoning Focus (RF)	Concrete security (CS)	Specification language
● – automation focus	● – security + efficiency	* – process calculus
● – expressiveness focus	● – security only	▷ – imperative
	○ – no support	◇ – functional

TABLE II

OVERVIEW OF TOOLS FOR COMPUTATIONAL SECURITY ANALYSIS. SEE SECTION II-D FOR MORE DETAILS ON COMPARISON CRITERIA.

II. Design-Level Security - Computational Tools

Discussion: Achievements



Achievements Machine-checked security for real-world cryptographic designs

Tool	Protocol / System	Property verified
CryptoVerif	TLS 1.3, Signal, WireGuard	Computational security
EasyCrypt	Amazon AWS key management	Computational security
EasyCrypt	SHA-3 standard	Functional + security
F7	miTLS (TLS 1.2)	Computational security at code level
F*	TLS 1.3 record layer	Security + implementation

II. Design-Level Security - Computational Tools

Discussion: Takeaways per Tool



EasyCrypt Closest to pen-and-paper cryptographic proofs.

- Supports relational program logic (pRHL): captures
 - ▶ **game-hopping**,
 - ▶ PRF/PRP switching lemma,
 - ▶ hybrid arguments.



III. Functional Correctness and Efficiency

III. Functional Correctness and Efficiency



- **Goal:** Bridge the gap between a **proven-secure design** and an **efficient, correct implementation**.
- Two requirements for implementations:
 - (1) **Functional correctness** — the implementation is an accurate translation of the design spec; same input/output behavior.
 - (2) **Efficiency** — performance must meet real deployment requirements.

III. Functional Correctness and Efficiency

Background: What is Functional Correctness?



An implementation can be proved functionally correct in two ways:

- (1) **Equivalence to a reference implementation** — the implementation produces the same outputs as a trusted specification for all valid inputs.
- (2) **Pre- and post-conditions** — the implementation satisfies:
 - **Pre-conditions:** what the program *requires* on inputs.
 - **Post-conditions:** what the program *guarantees* on outputs.

III. Functional Correctness and Efficiency

Background: Carrying Guarantees to Machine Code



- Source-level proofs must be carried down to machine code:

Approach	Idea	Examples
Verified compiler	Prove the compiler preserves semantics	CompCert (C)
Assembly-level tools	Verify the assembly directly	Jasmin, Vale
High-level + extraction	Prove in HOL/CIC, extract code	F*, Fiat Crypto

- **Trade-off:**

- Higher-level $\Rightarrow$ easier verification, *fewer* optimizations.
- Assembly-level $\Rightarrow$ best-in-class performance, *harder* verification.

III. Functional Correctness and Efficiency

Overview of Program Verification Tools (Table III)



Tool	Mem-safe	Auto	Param	Input lang	Target
Cryptol+SAW	•	•	○	C, Java	C, Java
Dafny	○	•	○	Dafny	C#, Java, Go
F*	•	•	○	OCaml, F#, C, Asm	C, Wasm
Fiat Crypto	•	○	•	Gallina	C
Jasmin	•	•	○	Jasmin	x86-64 Asm
Vale	•	•	○	Vale	x86-64/ARM Asm
Why3	•	○	○	WhyML	OCaml

• automated ○ interactive Param = parametric verification

III. Functional Correctness and Efficiency

A/S/T/U for Program Verification Tools



Accuracy (A) 1. Target(s)

- Source-level analysis
- Assembly-level analysis

Scope (S) 1. Memory-safety

- Bounds checking, null deref, use-after-free

Trust (T) 1. Trusted Computing Base

- SMT solvers, Coq, Isabelle, ...

Usability (U) 1. Automation 2. Parametric Verification 3. Input Language

	Jasmin	F*	Fiat
Target	Asm	C/Wasm	C
Mem-safe	•	•	•
Auto	•	•	○
Param	○	○	•
TCB	Coq, Dafny, Z3	self, SMT	Coq

III. Functional Correctness and Efficiency

A: Target Level of Analysis



Accuracy (A) 1. Target(s)

→ At what level is the analysis carried out?

Level	Guarantee	Challenge
Source (C/OCaml)	Proof about source code	Must use verified compiler to reach machine code
Assembly (x86/ARM)	Proof about actual binary	Harder to verify
Both	Strongest guarantee	Highest effort

III. Functional Correctness and Efficiency

S: Memory Safety & T: Trusted Computing Base



Scope (S) 1. Memory Safety

→ Can the tool verify that all runs are free of memory errors?

- Memory-safety errors include: **buffer overflow**, **null pointer dereference**, **use-after-free**.

Trust (T) 1. Trusted Computing Base (TCB)

→ What must be trusted for the result to be valid?

TCB component	Examples
Block verification (SMT)	Z3, CVC4
Interactive theorem prover	Coq (Rocq), Isabelle
Tool itself (self-certified)	F*, EasyCrypt

III. Functional Correctness and Efficiency

U: Automation, Parametric Verification, Input Language



Usability (U) 1. Automation

- **Fully automated** ($\bullet$): automatic proof search (e.g., Jasmin, F^*).
- **Semi-automated** ($\circ$): automated + interactive (e.g., Why3).
- **Interactive only** ($\circ$): full user guidance needed.

2. Parametric Verification

- Can the tool verify **generic, parameterized** implementations?
- Example: Fiat Crypto generates verified elliptic curve code parameterized by prime modulus, limb representation, and hardware platform.

3. Specification Language

- **Functional** (OCaml, Gallina): natural for algorithm specs.
- **Imperative** (C, Jasmin, Vale): natural for game-style, performance-tuned code.

III. Functional Correctness and Efficiency

Case Study: Curve25519 Implementations



- Compare verified & unverified Curve25519 implementations on scalar multiplication.
- Baseline: donna64 (unverified C). Metric: % faster (median, 5K executions).

Impl	FC	CT	Tools	Target	% faster
evercrypt	•	•	F*, Vale	64-bit C, Intel ADX	+25.92
precomp	○	○	—	Intel ADX asm	+25.77
sandy2x	○	○	—	Intel AVX asm	+11.15
hacl	•	•	F*	64-bit C	+8.69
jasmin	○	•	Jasmin	Intel x86-64 asm	+7.88
amd64	△	•	Coq, SMT	Intel x86-64 asm	+6.11
fiat	•	○	Fiat Crypto	64-bit C	+5.39
donna64	○	—	—	64-bit C (baseline)	0.00

FC = Functional Correctness verified; CT = Constant-time verified

III. Functional Correctness and Efficiency

Achievements & Takeaways



Achievements Verified primitives deployed at Internet-scale

- **HACL*** (F*, Vale) → Mozilla Firefox's NSS security engine.
- **Fiat Crypto** → Google's BoringSSL (elliptic curve code).
- **EverCrypt** → Zinc crypto library (Linux kernel).

Challenge Automating equivalence proofs & common arithmetic routines.

→ Verifying Montgomery arithmetic, field arithmetic from scratch for every project wastes effort; a *shared verified library* is needed.



IV. Conclusion

IV. Conclusion

Summary of Three Layers



Layer	Key tools	Achievement	Open challenge
Design-Level (Symbolic)	ProVerif, Tamarin	TLS 1.3, 5G, Signal verified	Unbounded equivalence beyond diff-equivalence
Design-Level (Computational)	CryptoVerif, EasyCrypt	Machine-checked proofs for TLS 1.3, AWS, SHA-3	Scalable UC composition; machine-checked UC nascent
Functional Correctness & Eff.	Jasmin, Vale F*, Fiat Crypto	Verified $\approx$ fastest unverified HACL* $\rightarrow$ Firefox, NSS	Common arithmetic library; automated equivalence proofs

IV. Conclusion

The “Grand Slam”



The “Grand Slam” of CAC

- Ideally, one implementation simultaneously satisfies **all four**:
 - (i) Provably secure in the computational model,
 - (ii) Correctly implemented by a reference implementation,
 - (iii) Reference $\equiv$ optimized implementation (functional correctness),
 - (iv) Optimized implementation is constant-time (side-channel safe).



V. Future Works

V. Future Works

Next Seminar: Formally Verifying Kyber



The next seminars will apply the CAC tools studied today to **ML-KEM** (CRYSTALS-Kyber, NIST FIPS-203):

- **Episode IV: Implementation Correctness**

[TCHES 2023]

- First formally verified implementations of **any post-quantum cryptosystem**.
- Kyber-768 in **Jasmin** (reference + AVX2): machine-checked functional correctness w.r.t. an **EasyCrypt** specification close to the NIST pseudocode.
- Performance: competitive with existing hand-optimized C/assembly.

- **Episode V: Machine-checked IND-CCA Security of ML-KEM**

[CRYPTO 2024]

- Machine-checked proof that **ML-KEM** is IND-CCA secure under the **MLWE** assumption, fully formalized in **EasyCrypt**.
- Covers: IND-CPA security of Kyber PKE → Fujisaki-Okamoto transform (ROM) → IND-CCA security of ML-KEM spec.
- Two constant-time Jasmin implementations proved functionally equivalent to the ML-KEM spec $\Rightarrow$ end-to-end verified, assembly-level IND-CCA secure KEM.

Together, Episodes IV & V realize the “grand slam” for a post-quantum KEM:

computational security + functional correctness + efficiency + constant-time.